



nextwork.org

Build a Three-Tier Web App

CH

Arpita Chowdhury

User Information

Get User Data

```
{
  "email": "test@example.com",
  "name": "Test User",
  "userId": "1"
}
```



Introducing Today's Project!

In this project, I will demonstrate how to design and connect a complete three-tier serverless web architecture on AWS, using S3 for frontend storage, CloudFront for global content delivery, API Gateway and Lambda for backend logic, and DynamoDB for data storage. I'm doing this project to learn how real-world cloud applications are built end to end, how different AWS services communicate securely with each other, and how serverless architectures enable scalability, high availability, and low operational overhead without managing servers.

Tools and concepts

Services I used were Amazon S3, Amazon CloudFront, AWS Lambda, Amazon API Gateway, and Amazon DynamoDB. Key concepts I learnt include Lambda functions for running backend logic without managing servers, API Gateway for exposing Lambda functions as secure REST APIs, DynamoDB for storing and retrieving data using a partition key, CloudFront for globally distributing web content, S3 for hosting static website files, and CORS configuration to allow secure communication between the frontend and backend across different domains.

Project reflection

This project took me approximately 3 hours. The most challenging part was debugging the integration between CloudFront, API Gateway, and Lambda, especially identifying CORS issues and incorrect API URLs across multiple services. It was most rewarding to see the three-tier architecture work end-to-end, with my CloudFront website successfully fetching and displaying data from DynamoDB through API Gateway and Lambda.

placeholder

.....



Presentation tier

For the presentation tier, I will set up Amazon S3 to host my website's static files and Amazon CloudFront to distribute the content globally, because this combination provides a fast, scalable, and cost-effective way to deliver a static website with low latency to users around the world.

I accessed my delivered website by using the CloudFront distribution URL, confirming that CloudFront was correctly pulling content from the S3 origin and serving it to end users with low latency.

User Information

☐



Logic tier

For the logic tier, I will set up an AWS Lambda function connected to API Gateway, because Lambda will act as the “brains” of my application by running backend code to fetch user data from a DynamoDB table, while API Gateway provides a public REST endpoint that can receive GET requests from the website and securely trigger the Lambda function to return the requested data.

The Lambda function retrieves data by reading the `userId` from the API Gateway query string parameters, using the AWS SDK's `GetCommand` to fetch the matching item from the `UserData` DynamoDB table, and returning the result as a JSON HTTP response with the appropriate status code (200 if found, 404 if not).



```
JS index.mjs x Welcome
JS index.mjs > ...
1 import { DynamoDBClient } from "@aws-sdk/client-dynamodb";
2 import { DynamoDBDocumentClient, GetCommand } from "@aws-sdk/lib-dynamodb";
3
4 const ddbClient = new DynamoDBClient({ region: 'us-east-1' }); // Make sure to replace this with your actual AWS region
5 const ddb = DynamoDBDocumentClient.from(ddbClient);
6
7 async function handler(event) {
8   const userId = String(event.userId); // Make sure to extract userId from the event
9   const params = {
10     TableName: 'UserData',
11     Key: { userId }
12   };
13
14   try {
15     const command = new GetCommand(params);
16     const data = await ddb.send(command); // Log the raw response from DynamoDB
17     console.log("DynamoDB Response:", JSON.stringify(data));
18     const { Item } = data;
19     if (Item) {
20       console.log("User data retrieved:", Item);
21       return Item;
22     } else {
23       console.log("No user data found for userId:", userId);
24       return null;
25     }
26   } catch (err) {
27     console.error("Unable to retrieve data:", err);
28     return err;
29   }
30 }
31
32 export { handler }; Amazon Q Tip 1/3: Start typing to get suggestions ([ESC] to exit)
```



Data tier

For the data tier, I will set up a DynamoDB table (e.g., UserData) to store and manage my application's user records, because DynamoDB is a fast, fully managed, serverless NoSQL database that integrates smoothly with Lambda and API Gateway, so my app can quickly store and retrieve data without managing servers.

The partition key for my DynamoDB table is `userId`, which means each item in the table is uniquely identified and evenly distributed based on the user's ID, allowing DynamoDB to quickly locate and retrieve a specific user's data with low latency.





Logic and Data tier

Once all three layers of my three-tier architecture are set up, the next step is to update my `script.js` to call my API Gateway endpoint and then display the returned DynamoDB data on my CloudFront-hosted website, because this connects the presentation layer to the logic + data layers, proving the entire system works end-to-end (website → API → Lambda → DynamoDB → back to the website).

To test my API, I opened the API Gateway invoke URL in my browser and passed a query string parameter (`?userId=1`), which triggers the GET method connected to my Lambda function. The results were a successful JSON response containing the user's data (`userId`, `name`, and `email`), which confirms that API Gateway correctly routed the request to Lambda, Lambda fetched the item from DynamoDB using the partition key, and the data flowed back through the API as expected.



Arpita Chowdhury

NextWork Student

nextwork.org

The screenshot shows a web browser window with the address bar displaying the URL: `as5umhxbz1.execute-api.us-east-1.amazonaws.com/prod/users?userId=1`. Below the address bar, there is a section labeled "Pretty-print" with a small square icon to its right. The main content area of the browser displays a JSON object: `{"email":"test@example.com","name":"Test User","userId":"1"}`.

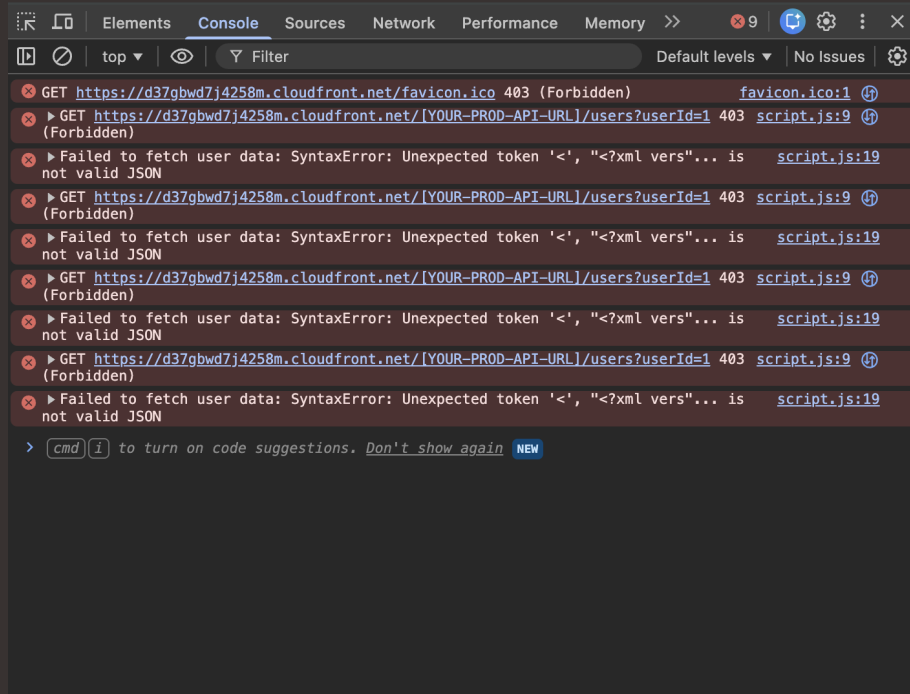


Console Errors

The error in my distributed site was because my `script.js` was still using the placeholder `[YOUR-PROD-API-URL]`, so the browser tried to fetch the API from my CloudFront domain instead of API Gateway. That request returned a 403 AccessDenied XML page, and then my code tried to parse it as JSON (`response.json()`), which caused the “Unexpected token ‘<’” JSON parsing error.

The error in my distributed site occurred because `script.js` was still using an incorrect or placeholder API URL, so CloudFront returned a 403 AccessDenied XML response, which JavaScript then failed to parse as JSON. I fixed this by updating `script.js` to use the correct API Gateway invoke URL and reuploading it to S3 so CloudFront could serve the updated file and successfully fetch user data from Lambda and DynamoDB.

I ran into a second error because my CloudFront website is calling API Gateway from a different domain, and the browser blocked the request due to missing/incorrect CORS headers (that’s why you see a 403 and the “Unexpected token ‘<’ ... not valid JSON” message—your code expected JSON but received an XML/HTML error page). To fix it, I enabled CORS for the `/users` GET method (and OPTIONS) in API Gateway, redeployed the stage, and ensured my Lambda/API response includes `Access-Control-Allow-Origin` (then refreshed/invalidated CloudFront to load the updated files).





Resolving CORS Errors

To resolve the CORS error, I first opened my API Gateway resource (e.g., /users) and confirmed the GET method existed, then used Actions → Enable CORS to allow browser requests (GET + OPTIONS) and set an allowed origin (like * or my CloudFront domain). After saving, I re-deployed the API to the prod stage, so the updated CORS headers would apply when my CloudFront website calls the API.

I also updated my Lambda function because my CloudFront website was calling my API from a different origin, and without CORS headers the browser blocks the response even if the API works. The changes I made were updating the handler to read the `userId` from `event.queryStringParameters.userId`, returning proper HTTP-style responses (`statusCode` + `body`) for API Gateway, and adding CORS response headers (especially `Access-Control-Allow-Origin: '*'`) to every success/error path so the browser is allowed to read the JSON response.

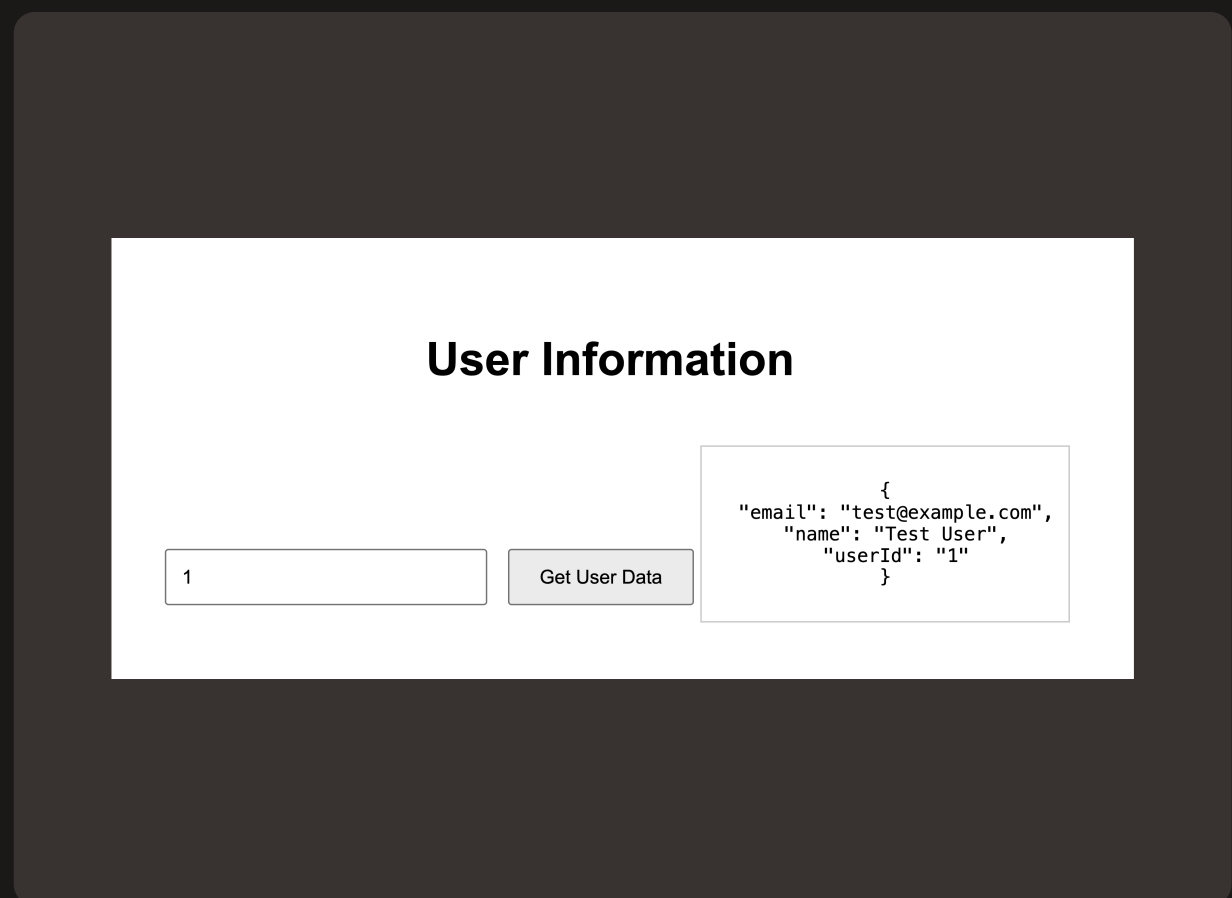


```
JS index.mjs X Welcome
JS index.mjs > ...
1 // Import individual components from the DynamoDB client package
2 import { DynamoDBClient } from "@aws-sdk/client-dynamodb";
3 import { DynamoDBDocumentClient, GetCommand } from "@aws-sdk/lib-dynamodb";
4
5 const ddbClient = new DynamoDBClient({ region: 'us-east-1' });
6 const ddb = DynamoDBDocumentClient.from(ddbClient);
7
8 async function handler(event) {
9   const userId = event.queryStringParameters.userId;
10   const params = {
11     TableName: 'UserData',
12     Key: { userId }
13   };
14
15   try {
16     const command = new GetCommand(params);
17     const { Item } = await ddb.send(command);
18
19     if (Item) {
20       return {
21         statusCode: 200,
22         headers: {
23           'Content-Type': 'application/json',
24           'Access-Control-Allow-Origin': '*' // Allow CORS for all origins, replace '*' with specific origin
25         },
26         body: JSON.stringify(Item)
27       };
28     } else {
29       return {
30         statusCode: 404,
31         headers: {
32           'Content-Type': 'application/json',
33           'Access-Control-Allow-Origin': '*'
34         },
35         body: JSON.stringify({ message: 'User not found' })
36       };
37     }
38   } catch (error) {
39     console.error('Error retrieving user data:', error);
40     return {
41       statusCode: 500,
42       headers: {
43         'Content-Type': 'application/json',
44         'Access-Control-Allow-Origin': '*'
45       },
46       body: JSON.stringify({ message: 'Internal server error' })
47     };
48   }
49 }
```




Fixed Solution

I verified the fixed connection between API Gateway and CloudFront by refreshing my CloudFront URL, entering a valid user ID, clicking Get User Data, and confirming that the user information was successfully fetched from the API and displayed on the website without any CORS or console errors.





nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

